

UTS
PENGOLAHAN CITRA



NAMA : Muhammad Ragil Azra Lubis

NIM : 202431122

KELAS : F

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC :

ASISTEN : 1. Erin Katholica Sakrally Putri Marrison

2. Morgan Immanuel Nainggolan

3.

4.

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

DAFTAR ISI

DAFTAR ISI	2
BAB I PENDAHULUAN	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah	3
1.3 Manfaat Masalah	3
BAB II LANDASAN TEORI	4
2.1 Representasi Citra Digital Dan Ruang Warna RGB.....	4
2.2 Deteksi Warna Dan Penerapan Thresholding	4
2.3 Histogram Citra Dan Peranannya Dalam Analisis	5
2.5 Operasi Titik Untuk Kecerahan Dan Kontras.....	6
2.6 Peningkatan Kualitas Citra (Image Enhancement) Pada Kasus Backlight.....	6
BAB III HASIL	8
BAB IV PENUTUP.....	21
DAFTAR PUSTAKA	22

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana cara mengisolasi dan mendeteksi warna spesifik (merah, hijau, dan biru) pada citra digital berupa tulisan tangan menggunakan metode *thresholding* pada ruang warna HSV?
2. Bagaimana pengaruh proses ekstraksi saluran warna (RGB) dan analisis distribusi intensitas piksel melalui grafik histogram dalam memahami komposisi sebuah citra digital?
3. Bagaimana langkah yang tepat untuk memperbaiki kualitas visual citra grayscale yang mengalami masalah pencahayaan belakang (*backlight*) menggunakan algoritma operasi titik dasar (penyesuaian kecerahan dan kontras)?

1.2 Tujuan Masalah

1. Melakukan deteksi warna merah, hijau, dan biru pada citra tulisan tangan menggunakan metode *thresholding* berbasis ruang warna HSV.
2. Menganalisis hasil deteksi warna serta histogram masing-masing kanal untuk menentukan nilai ambang batas yang optimal.
3. Memperbaiki kualitas citra *backlight* melalui serangkaian operasi piksel, meliputi konversi ke grayscale, pencerahan, peningkatan kontras, serta kombinasi keduanya, sehingga subjek yang gelap menjadi lebih menonjol dibandingkan latar belakang yang terang.
4. Membandingkan hasil pemrosesan secara visual dalam satu figure untuk mengevaluasi efektivitas setiap teknik yang diterapkan.

1.3 Manfaat Masalah

1. Memberikan pemahaman komprehensif terkait cara kerja dan keunggulan ruang warna HSV dibandingkan model RGB dalam memisahkan intensitas cahaya dari warna murni.
2. Meningkatkan kemampuan analitis mahasiswa dalam mengkalibrasi parameter nilai piksel digital melalui observasi histogram secara mendalam.
3. Membekali mahasiswa dengan teknik fundamental Peningkatan Kualitas Citra (*Image Enhancement*) yang dapat langsung diaplikasikan untuk menyelesaikan masalah nyata, seperti memperbaiki dokumen pudar atau menyempurnakan foto dengan kondisi pencahayaan buruk (*backlight*).

BAB II

LANDASAN TEORI

2.1 Representasi Citra Digital Dan Ruang Warna RGB

Citra digital terdiri dari bagian-bagian kecil yang disebut piksel. Setiap piksel menunjukkan seberapa terang cahaya pada lokasi tertentu. Secara matematis, citra bisa digambarkan sebagai fungsi dua dimensi $f(x,y)$ di mana x dan y menunjukkan posisi piksel, dan nilai f di titik itu menunjukkan tingkat kecerahan atau intensitas. Pengubahan citra dari bentuk kontinu menjadi format digital dilakukan melalui dua langkah, yaitu pencuplikan dan kuantisasi. Pencuplikan menentukan seberapa detail citra dengan membagi area citra menjadi grid, sedangkan kuantisasi mengatur jumlah tingkat intensitas yang ada. Dalam citra 8-bit, terdapat 256 tingkat keabuan dari 0 (hitam total) hingga 255 (putih total).

Untuk menghasilkan warna, digunakan model ruang warna RGB yang merupakan model aditif. Dalam model ini, setiap piksel memiliki tiga bagian intensitas: merah, hijau, dan biru. Dengan mencampurkan ketiga warna tersebut dalam berbagai proporsi, akan muncul beragam warna yang dapat dilihat oleh mata manusia. Karena setiap bagian menyimpan informasi secara terpisah, pemisahan warna menjadi langkah awal yang sangat penting dalam banyak aplikasi pengolahan citra, terutama untuk mendeteksi warna tertentu. Memisahkan warna memungkinkan analisis terhadap distribusi intensitas masing-masing warna dalam citra dan memudahkan untuk menetapkan batas saat ingin memisahkan objek berwarna tertentu dari latar belakang.

2.2 Deteksi Warna Dan Penerapan Thresholding

Deteksi warna dalam pengolahan gambar digital yang sederhana biasanya menggunakan metode ambang atau thresholding. Metode ini mengubah gambar dengan banyak lapisan menjadi gambar biner dengan cara menentukan sebuah nilai batas. Piksel yang intensitasnya lebih rendah atau lebih tinggi dari batas tersebut akan dianggap sebagai objek, sedangkan sisanya akan dianggap latar belakang. Untuk mendeteksi beberapa warna secara spesifik, digunakan teknik thresholding rentang. Teknik ini menentukan batas bawah dan batas atas untuk setiap saluran warna yang relevan. Piksel yang memenuhi semua syarat yang telah ditentukan akan diberi nilai 1 sebagai objek, sedangkan yang tidak memenuhi akan mendapat nilai 0 sebagai latar belakang. Menentukan nilai ambang sangat penting. Jika batas terlalu sempit, bagian dari objek bisa hilang, sementara jika terlalu lebar, area latar belakang atau noise bisa masuk.

Pemilihan batas ini bisa dilakukan secara manual dengan melihat histogram dari setiap saluran warna. Dalam kondisi ideal seperti tulisan tangan di kertas putih, tinta warna akan menunjukkan nilai puncak yang cukup kontras dengan latar belakang putih. Dengan memperhatikan letak puncak di histogram, batas bawah dan atas dapat ditentukan agar hanya piksel tinta yang terdeteksi. Walaupun ada metode otomatis seperti Otsu, penentuan secara manual seringkali lebih akurat untuk kasus yang memiliki jumlah objek terbatas dan distribusi intensitas yang sederhana dengan dua puncak.

2.3 Histogram Citra Dan Peranannya Dalam Analisis

Histogram adalah grafik yang menunjukkan seberapa sering setiap tingkat intensitas muncul dalam gambar. Untuk gambar hitam-putih, histogram hanya memiliki satu saluran, sedangkan untuk gambar RGB, histogram bisa ditampilkan secara terpisah untuk saluran merah, hijau, dan biru. Bentuk histogram memberikan informasi penting tentang karakteristik kontras, kecerahan, dan komposisi intensitas dari suatu gambar. Histogram yang mengumpul di sisi kiri menunjukkan gambar yang gelap, sementara yang di sisi kanan menunjukkan gambar yang terlalu terang, dan histogram yang tersebar merata menunjukkan kontras yang baik.

Dalam konteks mendeteksi warna tinta pada kertas putih, histogram untuk setiap saluran akan menunjukkan dua kelompok utama. Latar putih yang memantulkan seluruh spektrum cahaya akan menghasilkan puncak tinggi di daerah intensitas tinggi pada semua saluran. Tinta dengan warna tertentu hanya akan membentuk puncak yang lebih rendah di daerah intensitas rendah hingga menengah pada saluran yang sesuai dengan warna itu. Informasi ini membuktikan efektivitas metode thresholding berbasis rentang. Dengan memeriksa histogram, pengguna bisa memastikan bahwa batas ambang yang dipilih benar-benar memisahkan antara tinta dan latar belakang tanpa ada tumpang tindih yang signifikan.

2.4 Konversi Citra Berwarna Ke Grayscale

Banyak teknik pengolahan gambar lebih mudah dilakukan pada gambar monokrom karena hanya memiliki satu saluran intensitas. Mengubah gambar RGB menjadi grayscale dilakukan dengan menambahkan ketiga saluran dengan menggunakan koefisien bobot yang mewakili seberapa peka mata manusia terhadap warna. Mata manusia paling peka terhadap warna hijau, kemudian merah, dan paling sedikit peka terhadap biru. Oleh karena itu, bobot standar yang umum digunakan adalah 0,299 untuk merah, 0,587 untuk hijau, dan 0,114 untuk biru. Rumus ini menghasilkan gambar abu-abu yang tingkat kecerahannya mirip dengan penglihatan alami kita.

Hasil dari konversi ini disebut gambar grayscale dan hanya memiliki satu nilai intensitas untuk setiap piksel. Dengan hanya satu saluran, kompleksitas perhitungan berkurang, dan analisis seperti

peningkatan kontras, perataan histogram, atau deteksi tepi menjadi lebih mudah. Dalam proyek yang melibatkan perbaikan gambar backlight, mengubah gambar menjadi grayscale terlebih dahulu mempermudah penerapan operasi titik atau adaptif tanpa khawatir merusak keseimbangan warna. Fokus perbaikan kemudian sepenuhnya pada peningkatan detail berdasarkan informasi cahaya yang ada.

2.5 Operasi Titik Untuk Kecerahan Dan Kontras

Peningkatan kecerahan dan kontras adalah proses sederhana yang mengubah setiap piksel tanpa mempedulikan piksel lain di sekitarnya. Kecerahan ditambah dengan suatu angka positif ke semua nilai cahaya, sedangkan kontras diubah dengan mengalikan setiap piksel dengan angka tertentu. Gabungan dari kedua proses ini disebut transformasi linear: $s = a \cdot r + c$, dengan r intensitas masukan, s intensitas keluaran, a faktor kontras, dan c konstanta kecerahan. Jika hanya menambahkan, seluruh gambar akan terlihat lebih cerah, termasuk latar belakang yang sudah terang. Sebaliknya, jika hanya mengalikan, bagian gelap akan tetap gelap dan bagian terang semakin menonjol. Untuk memperbaiki gambar dengan pencahayaan belakang secara keseluruhan, diperlukan pemilihan a dan c yang tepat agar bagian gelap mendapatkan peningkatan yang cukup tanpa mengorbankan detail latar belakang. Namun, metode ini memiliki batasan karena tidak bisa membedakan antara subjek dan latar belakang dengan tepat. Karena itu, pada gambar dengan perbedaan pencahayaan yang sangat besar, teknik yang lebih adaptif menjadi pilihan yang lebih baik.

2.6 Peningkatan Kualitas Citra (Image Enhancement) Pada Kasus Backlight

Peningkatan kualitas citra adalah serangkaian proses yang bertujuan untuk memperbaiki detail visual dari gambar agar lebih mudah untuk dimengerti, baik oleh manusia maupun mesin analisis. Salah satu masalah umum yang sering dihadapi pada tahap ini adalah silau dari belakang atau backlight. Situasi backlight terjadi ketika sumber cahaya utama berada di belakang objek yang difoto dan menghadap ke arah kamera. Karena kemampuan sensor kamera dalam menangkap cahaya terbatas dibandingkan dengan mata manusia, lensa kamera secara otomatis akan mengurangi pencahayaan yang masuk agar latar belakang yang terang tidak kehilangan detail. Namun, ini menyebabkan objek di depan yang berlawanan dengan cahaya menjadi tampak sangat gelap seperti siluet.

Untuk mengatasi masalah penurunan kualitas gambar akibat backlight, diperlukan algoritma yang melakukan modifikasi pada piksel, baik secara linier maupun non-linier. Proses dasar yang biasa digunakan adalah penambahan kecerahan dan perbaikan kontras. Penyesuaian kecerahan dilakukan dengan menambah atau mengurangi angka tetap pada seluruh nilai intensitas gambar, yang akan memindahkan kurva histogram ke arah nilai terang. Sementara itu, untuk meningkatkan kontras, dilakukan pengalihan pada nilai piksel, sehingga jarak antara piksel gelap dan piksel terang pada siluet semakin lebar. Ini memungkinkan bagian wajah atau tubuh objek yang sebelumnya berada di area gelap histogram untuk terlihat lebih jelas, sehingga detail tekstur baju atau bentuk wajah dapat dilihat dengan lebih baik.

Halangan utama yang harus diperhatikan dalam penerapan algoritma untuk meningkatkan kecerahan dan mengatur kontras ini adalah fenomena clipping atau pembakaran warna. Clipping secara matematis terjadi ketika hasil perbaikan nilai matriks melebihi batas maksimum penyimpanan bit memori, yaitu lebih dari 255. Piksel-piksel yang lewati batas ini akan terpaksa menjadi 255, yang menyebabkan hilangnya gradasi tekstur menjadi warna putih yang solid atau warna dasar yang teramat ter saturasi secara artifisial. Oleh sebab itu, rasio dan konstanta dalam fungsi untuk meningkatkan tampilan siluet gelap pada gambar backlight harus dirancang dan diukur dengan hati-hati agar keseimbangan antara mencerahkan objek di depan dan mempertahankan keutuhan visual di belakang tetap terjaga tanpa menyebabkan artefak warna yang berlebihan.

BAB III

HASIL

PENJELASAN CODE NO 1

Import Library

```
] :  ► import cv2
      import numpy as np
      import matplotlib.pyplot as plt
```

Disini saya memakai kode import cv2 untuk memanggil library OpenCV. Library ini saya gunakan karena OpenCV adalah pustaka utama untuk pengolahan citra digital. Fungsinya nanti akan saya pakai untuk membaca file gambar, mengubah warnanya, melakukan thresholding, resize, dan lain-lain. Kemudian saya juga menulis import matplotlib.pyplot as plt. Matplotlib ini saya gunakan untuk menampilkan gambar di Jupyter Notebook, lalu saya mengimpor library NumPy dan memberi alias np. NumPy adalah pustaka untuk komputasi numerik.

1. A

```
In [18]: ► img = cv2.imread('Nama.jpeg')
          rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

Baris pertama, `img = cv2.imread('Nama.jpeg')`, berfungsi membaca file citra bernama `Nama.jpeg` dari direktori kerja dan menyimpannya ke dalam variabel `img`. Baris kedua `rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` mengonversi citra dari ruang warna BGR ke RGB. Konversi ini diperlukan agar saat citra ditampilkan menggunakan `matplotlib.pyplot.imshow`, warna yang muncul sesuai dengan aslinya.

```
In [19]: ► R = rgb[:, :, 0]
          G = rgb[:, :, 1]
          B = rgb[:, :, 2]
```

Kode ini digunakan untuk memisahkan citra `rgb` (yang sudah berada dalam ruang warna Red-Green-Blue) menjadi tiga kanal tunggal. Citra `rgb` adalah larik NumPy tiga dimensi berukuran (baris, kolom, 3). Indeks terakhir (0, 1, 2) menunjuk pada kanal warna tertentu: 0 untuk Merah, 1 untuk Hijau, dan 2 untuk Biru. Dengan notasi slicing `[:, :, 0]`, kita mengambil semua baris (: pertama) dan semua kolom (: kedua) hanya dari kanal ke-0. Dengan demikian `R` berisi nilai intensitas kanal merah, `G` berisi intensitas kanal hijau, dan `B` berisi intensitas kanal biru.


```
In [28]: ► fig, axs = plt.subplots(2, 2, figsize=(12, 10))
axs[0, 0].imshow(rgb)
axs[0, 0].set_title("Citra Asli")

axs[0, 1].imshow(B, cmap='gray')
axs[0, 1].set_title("Kanal Biru")

axs[1, 0].imshow(R, cmap='gray')
axs[1, 0].set_title("Kanal Merah")

axs[1, 1].imshow(G, cmap='gray')
axs[1, 1].set_title("Kanal Hijau")

plt.tight_layout()
plt.show()
```

fig, axs = plt.subplots(2, 2, figsize=(12, 10)) Membuat figure baru dengan grid subplot 2×2. Variabel **axs** adalah array 2×2 yang menyimpan empat objek sumbu, Ukuran kanvas diatur 12×10 inci agar setiap gambar cukup besar.

axs[0, 0].imshow(rgb) Citra Asli Subplot di posisi kiri atas menampilkan citra rgb dengan warna penuh. Ini berfungsi sebagai acuan visual awal untuk melihat tulisan tangan berwarna merah, hijau, dan biru di atas kertas putih.

axs[0, 1].imshow(B, cmap='gray') Kanal Biru Subplot kanan atas menampilkan kanal biru B hasil ekstraksi `rgb[:, :, 2]` sebagai citra grayscale. Area yang tampak terang pada kanal ini menunjukkan bagian citra yang memiliki komponen biru dominan yaitu tinta biru.

axs[1, 0].imshow(R, cmap='gray') Kanal Merah Subplot kiri bawah menampilkan kanal merah R `rgb[:, :, 0]`. Tinta merah dan latar putih akan tampak terang, sedangkan tinta biru dan hijau tampak lebih gelap.

axs[1, 1].imshow(G, cmap='gray') Kanal Hijau Subplot kanan bawah menampilkan kanal hijau G `rgb[:, :, 1]`. Tinta hijau akan menyala terang, sementara tinta merah dan biru cenderung gelap.

axs[...].set_title(...) Masing-masing subplot diberi judul deskriptif agar mudah dikenali.

plt.tight_layout() Mengatur tata letak agar tidak ada elemen yang saling bertumpuk (misalnya judul dengan gambar di atasnya).

plt.show() Merender dan menampilkan figure secara keseluruhan.

```

fig, axs = plt.subplots(2, 2, figsize=(15, 10))
axs[0, 0].set_title("Histogram Citra Asli (RGB)")
color = ('r', 'g', 'b')
for i, col in enumerate(color):
    hist = cv2.calcHist([rgb], [i], None, [256], [0, 256])
    axs[0, 0].plot(hist, color=col)
    axs[0, 0].set_xlim([0, 256])

# Histogram Kanal Biru
axs[0, 1].hist(B.ravel(), 256, [0, 256], color='blue', alpha=0.7)
axs[0, 1].set_title("Histogram Kanal Biru")

# Histogram Kanal Merah
axs[1, 0].hist(R.ravel(), 256, [0, 256], color='red', alpha=0.7)
axs[1, 0].set_title("Histogram Kanal Merah")

# Histogram Kanal Hijau
axs[1, 1].hist(G.ravel(), 256, [0, 256], color='green', alpha=0.7)
axs[1, 1].set_title("Histogram Kanal Hijau")

plt.tight_layout()
plt.show()

```

fig, axs = plt.subplots(2, 2, figsize=(15, 10)) Membuat figure baru dengan grid subplot 2×2. Variabel **axs** adalah array 2×2 yang menyimpan empat objek sumbu, Ukuran kanvas diatur 15×10 inci agar setiap gambar cukup besar.

color = ('r', 'g', 'b') mendefinisikan warna garis untuk kanal Merah, Hijau, dan Biru. Perulangan **for i, col in enumerate(color):** mengiterasi ketiga kanal. **cv2.calcHist([rgb], [i], None, [256], [0, 256])** menghitung histogram untuk kanal ke-*i* (0=Merah, 1=Hijau, 2=Biru) dari citra **rgb**. Parameter **[256]** dan **[0, 256]** berarti histogram memiliki 256 bin yang mencakup rentang intensitas 0–255. **axs[0, 0].plot(hist, color=col)** menggambar histogram sebagai garis berwarna sesuai kanal. **set_xlim([0, 256])** membatasi sumbu *x* agar sesuai dengan rentang intensitas.

axs[0, 1].hist(B.ravel(), 256, [0, 256], color='blue', alpha=0.7) menampilkan histogram kanal biru menggunakan **plt.hist**. Data kanal **B** yang berupa larik 2D diubah menjadi 1D dengan **.ravel()**. Parameter **alpha=0.7** membuat batang histogram sedikit transparan untuk estetika. Pola yang sama diterapkan untuk kanal merah (**R**) di **axs[1, 0]** dan kanal hijau (**G**) di **axs[1, 1]**.

AMBANG BATAS

```

hsv = cv2.cvtColor(rgb, cv2.COLOR_RGB2HSV)

# Menentukan rentang ambang batas HSV untuk tiap warna
lower_blue = np.array([100, 50, 50])
upper_blue = np.array([140, 255, 255])

lower_red1 = np.array([0, 50, 50])
upper_red1 = np.array([10, 255, 255])
lower_red2 = np.array([170, 50, 50])
upper_red2 = np.array([180, 255, 255])

lower_green = np.array([35, 40, 40])
upper_green = np.array([90, 255, 255])

# Proses Masking berdasarkan nilai ambang batas
mask_blue = cv2.inRange(hsv, lower_blue, upper_blue)
mask_red1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask_red2 = cv2.inRange(hsv, lower_red2, upper_red2)
mask_red = cv2.add(mask_red1, mask_red2)
mask_green = cv2.inRange(hsv, lower_green, upper_green)

```

Baris pertama mengonversi citra dari ruang warna RGB ke HSV (*Hue, Saturation, Value*).

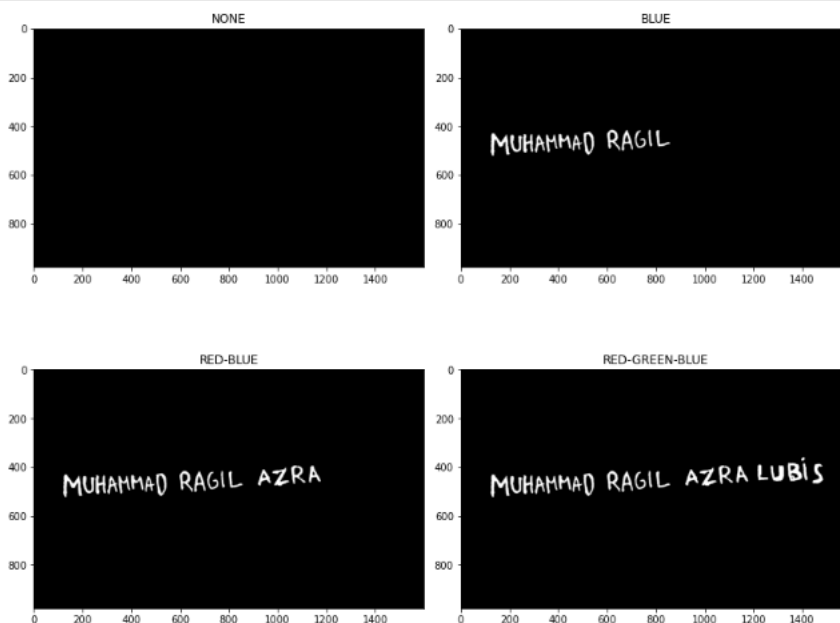
Rentang ambang batas didefinisikan dalam format [H, S, V]. Untuk warna biru, Hue berada di sekitar 100–140 derajat. Untuk hijau, Hue berkisar 35–90. Warna merah memerlukan dua rentang karena Hue adalah nilai sudut melingkar (0–360 derajat, dipetakan ke 0–180 di OpenCV). Merah terletak di dekat 0 derajat dan juga di dekat 360 derajat (yaitu 180 di OpenCV). Oleh karena itu, dua rentang digunakan: Hue 0–10 dan 170–180.

Fungsi `cv2.inRange` memeriksa setiap piksel pada citra HSV. Piksel yang nilai H, S, dan V-nya semuanya berada dalam batas yang ditentukan akan diisi 255 (putih) di citra keluaran, sedangkan di luar itu diisi 0 (hitam). Hasilnya adalah masker biner untuk masing-masing warna.

Karena warna merah terdiri dari dua rentang Hue yang terpisah, dibuat dua masker (`mask_red1` dan `mask_red2`) yang kemudian digabung menggunakan `cv2.add`. Operasi penjumlahan biner ini menghasilkan masker tunggal `mask_red` yang mencakup seluruh piksel merah, baik yang berada di sekitar sudut 0° maupun 360°. Masker biru dan hijau cukup dibuat dengan satu panggilan `inRange` karena rentang Hue-nya tidak terpotong oleh batas 180 derajat.

Menampilkan Hasil

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes[0, 0].imshow(np.zeros_like(mask_blue), cmap='gray')
axes[0, 0].set_title("NONE")
axes[0, 1].imshow(mask_blue, cmap='gray')
axes[0, 1].set_title("BLUE")
axes[1, 0].imshow(cv2.bitwise_or(mask_red, mask_blue), cmap='gray')
axes[1, 0].set_title("RED-BLUE")
axes[1, 1].imshow(cv2.bitwise_or(cv2.bitwise_or(mask_red, mask_blue), mask_green), cmap='gray')
axes[1, 1].set_title("RED-GREEN-BLUE")
plt.tight_layout()
plt.show()
```



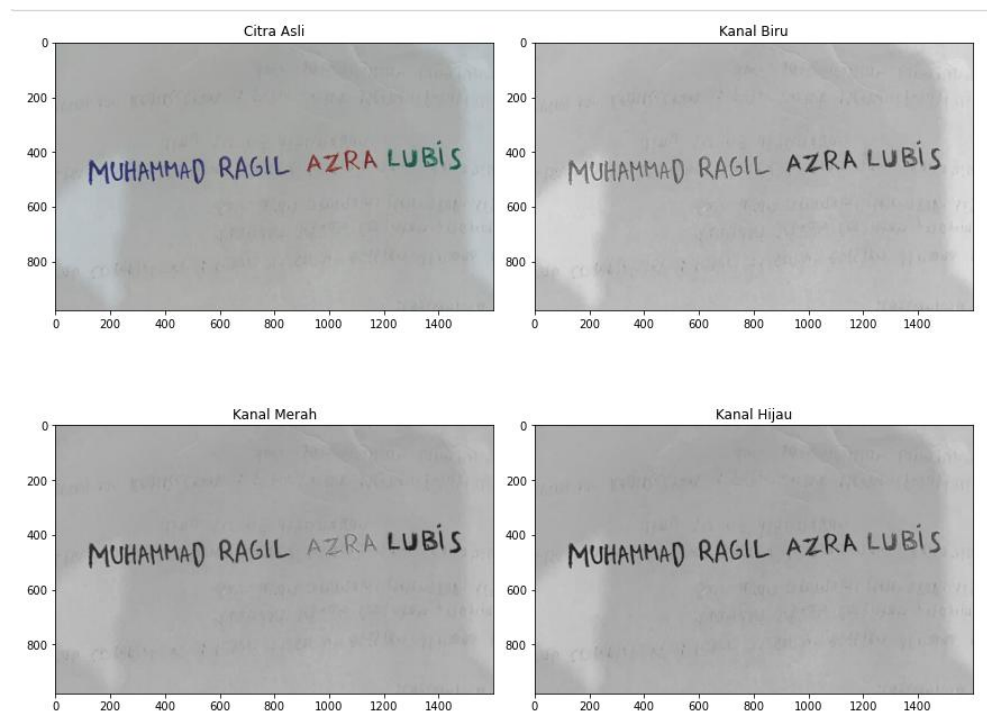
`axes[0, 0].imshow(np.zeros_like(mask_blue), cmap='gray')` menampilkan citra hitam pekat yang dibuat dengan `np.zeros_like(mask_blue)`.

`axes[0, 1].imshow(mask_blue, cmap='gray')` menampilkan citra biner hasil deteksi tinta biru menggunakan `cv2.inRange` pada ruang HSV. Subplot ini menunjukkan hasil deteksi tunggal warna biru, memperlihatkan bagian tulisan yang menggunakan tinta biru.

`axs[1, 0].imshow(cv2.bitwise_or(mask_red, mask_blue), cmap='gray')` menampilkan hasil operasi OR antara masker merah dan masker biru. `cv2.bitwise_or` menggabungkan dua citra biner sehingga piksel yang terdeteksi di salah satu atau kedua masker akan bernilai 255. Dengan demikian, subplot ini menampilkan semua piksel yang terdeteksi sebagai merah atau biru

`axs[1,1].imshow(cv2.bitwise_or(cv2.bitwise_or(mask_red,mask_blue),mask_green), cmap='gray')` menggabungkan ketiga masker (merah, biru, dan hijau) menggunakan operasi OR bertingkat. Hasilnya adalah masker komposit yang menampilkan seluruh piksel tinta yang terdeteksi, terlepas dari warnanya. Idealnya, semua tulisan tangan akan tampak sebagai area putih, menunjukkan bahwa sistem berhasil mendeteksi ketiga warna tinta.

ANALISIS



1. Citra Asli

Ini adalah gambar sumber yang menampilkan tulisan tangan di atas kertas putih. Tulisan tersebut menggunakan beberapa warna tinta yang berbeda:

"MUHAMMAD RAGIL": Ditulis dengan tinta berwarna biru.

"AZRA": Ditulis dengan tinta berwarna merah.

"LUBIS": Ditulis dengan tinta berwarna hijau.

Latar Belakang: Kertas berwarna putih/krem terang, yang secara teori memantulkan semua spektrum warna cahaya (merah, hijau, dan biru).

2. Kanal Merah (Red Channel)

Citra ini merepresentasikan intensitas warna merah pada gambar. Dalam representasi grayscale (hitam putih), warna putih/terang berarti intensitas merahnya tinggi, sedangkan hitam/gelap berarti intensitas merahnya rendah.

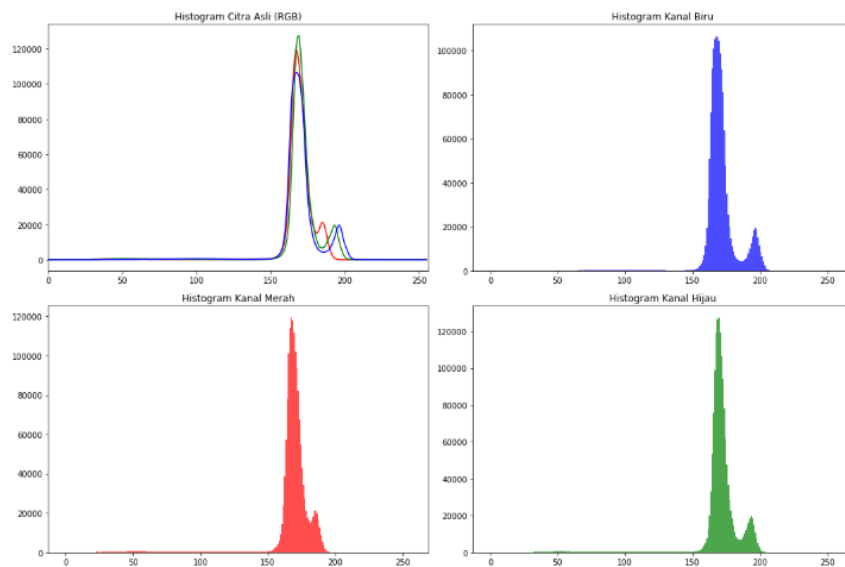
Kata "AZRA" (Tinta Merah): Terlihat sangat pudar dan hampir menyatu dengan warna latar belakang putih. Hal ini terjadi karena tinta merah memantulkan banyak cahaya merah, sehingga pikselnya memiliki nilai intensitas yang sangat tinggi mendekati 255 atau putih. Biru Dan Hijau Terlihat sangat gelap/hitam pekat karena tinta biru dan hijau menyerap cahaya merah, sehingga intensitas warna merah pada area tersebut sangat rendah.

3. Kanal Biru

Citra ini merepresentasikan intensitas warna biru. Kata "MUHAMMAD RAGIL" (Tinta Biru) Terlihat lebih terang (berwarna abu-abu) dibandingkan kata lainnya. Ini membuktikan bahwa area ini memiliki kandungan warna biru yang lebih dominan. Kata "AZRA" (Merah) dan "LUBIS" (Hijau): Terlihat sangat gelap dan tegas karena objek tersebut menyerap sebagian besar spektrum cahaya biru, sehingga nilai intensitas birunya sangat rendah.

4. Kanal Hijau

Berdasarkan hasil ekstraksi kanal warna, terdapat fenomena optis yang menarik pada matriks kanal hijau. Idealnya, tinta berwarna hijau akan memantulkan intensitas cahaya hijau yang tinggi sehingga akan tampak memudar dan menyatu dengan latar belakang kertas putih, sama halnya seperti perilaku tinta merah pada kanal merah dan tinta biru pada kanal biru. Namun, pada hasil observasi huruf yang ditulis menggunakan tinta hijau tetap terlihat gelap pada representasi citra kanal hijau. Anomali ini disebabkan oleh karakteristik pigmen fisis dari pena hijau yang digunakan, yang cenderung berwarna hijau gelap.



1. Histogram Citra Asli (RGB)

Pada bagian Histogram Citra Asli (RGB), grafik ini menampilkan kurva gabungan dari ketiga kanal warna sekaligus dalam satu sumbu. Sumbu horizontal menunjukkan nilai intensitas piksel dari rentang gelap hingga terang, sedangkan sumbu vertikal menunjukkan frekuensi kemunculan piksel. Karakteristik utama dari grafik ini adalah tumpang tindih kurva merah, hijau, dan biru yang membentuk puncak dominan pada rentang intensitas 160 hingga 180. Kesamaan pola dari ketiga kurva ini mengindikasikan bahwa sebagian besar area gambar, yakni latar belakang kertas, memiliki komposisi warna yang relatif netral atau mendekati spektrum abu-abu terang.

2. Histogram Kanal Biru

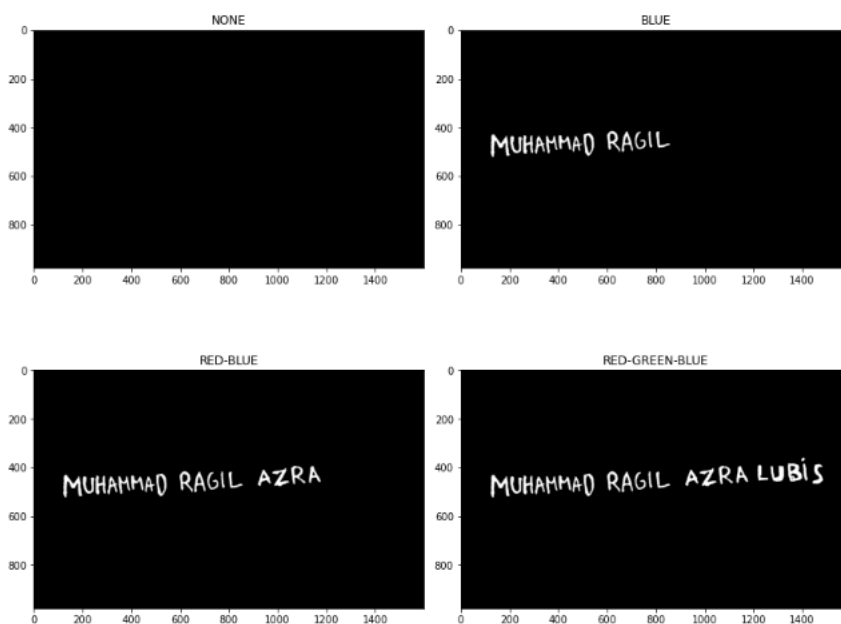
Histogram Kanal Biru secara spesifik mengisolasi sebaran piksel berdasarkan intensitas spektrum warna biru. Puncak utama pada grafik ini terpusat di sekitar nilai intensitas 170, yang mewakili warna dominan dari latar belakang kertas putih tersebut pada spektrum biru. Terdapat pula puncak sekunder yang lebih kecil di kisaran nilai 195, yang kemungkinan besar merepresentasikan area kertas yang memantulkan cahaya lebih kuat. Piksel-piksel pembentuk teks yang ditulis dengan tinta merah dan hijau, yang sifatnya menyerap cahaya biru, tersebar dengan frekuensi yang sangat rendah di area intensitas gelap pada sisi kiri grafik hingga membentang mendekati angka nol.

3. Histogram Kanal Merah

Histogram Kanal Merah, grafik ini memperlihatkan distribusi intensitas khusus untuk spektrum cahaya merah. Puncak utamanya berada pada rentang intensitas antara 165 hingga 175, dengan puncak sekunder yang terbentuk di sekitar nilai 185 hingga 190. Pola yang sedikit bergeser posisinya dibandingkan dengan kanal biru ini menunjukkan bagaimana sensor kamera secara spesifik menangkap pantulan cahaya merah dari permukaan kertas. Area teks bertinta biru dan hijau akan menyerap spektrum merah ini secara signifikan, sehingga piksel-piksel dari teks tersebut kehilangan intensitas terangnya dan jatuh pada rentang nilai yang sangat rendah di sebelah kiri kurva utama.

4. Histogram Kanal Hijau

Histogram Kanal Hijau merepresentasikan frekuensi intensitas warna hijau di seluruh bagian citra. Grafik ini menampilkan puncak utama yang sangat tajam di sekitar nilai 170, didampingi oleh puncak sekunder pada kisaran nilai 195. Menariknya, jika dibandingkan dengan puncak pada kanal individual merah dan biru, puncak pada kanal hijau ini terlihat memanjang paling tinggi ke atas. Hal ini mengisyaratkan bahwa komponen warna hijau memiliki intensitas yang sedikit lebih kuat pada komposisi cahaya yang terpantul dari latar belakang kertas. Sama halnya dengan prinsip sebelumnya, teks dengan tinta merah dan biru yang secara alami menyerap warna hijau akan menduduki rentang intensitas gelap yang mendatar di sisi paling kiri dari grafik tersebut.



1. Batas Ambang Biru

Nilai ambang batas untuk komponen Hue pada warna biru ditetapkan antara 100 hingga 140. Dalam sistem OpenCV, skala derajat warna yang aslinya berkisar dari 0 hingga 360 derajat dikompresi menjadi skala 0 hingga 179 agar sesuai dengan format *byte*. Pada skala OpenCV ini, rentang nilai 100 hingga 140 secara matematis merepresentasikan spektrum cahaya biru, sehingga rentang ini sangat ideal untuk mendeteksi tinta biru secara presisi tanpa menangkap warna-warna yang berdekatan seperti cyan atau ungu.

2. Batas Ambang Merah

Penanganan ambang batas untuk warna merah memerlukan metode yang berbeda dengan memecah rentang Hue menjadi dua bagian terpisah, yaitu 0-10 dan 170-180. Pendekatan ini merupakan solusi atas karakteristik bentuk silinder dari ruang warna HSV, di mana spektrum warna merah secara natural terletak di titik awal lingkaran (0 derajat) dan terus menyambung hingga ke titik akhir (360 derajat). Konsekuensinya pada sistem OpenCV, data piksel warna merah akan terdistribusi di kedua ujung skala batas Hue. Oleh karena itu, kedua hasil masking dari rentang batas bawah dan batas atas tersebut wajib dijumlahkan untuk mendapatkan representasi objek berwarna merah secara utuh pada citra.

3. Batas Ambang Biru

Ambang batas untuk deteksi warna hijau dialokasikan pada rentang Hue 35 hingga 90 yang merupakan pemetaan langsung dari spektrum hijau pada roda warna komputasi. Penyesuaian khusus dilakukan pada batas bawah parameter Saturation dan Value yang diturunkan sedikit menjadi 40. Penyesuaian ini berfungsi untuk meningkatkan toleransi algoritma terhadap tinta hijau yang memiliki tampilan visual yang lebih pudar atau memiliki tingkat pantulan cahaya yang lebih rendah dibandingkan tinta warna merah dan biru pada gambar tulisan tangan saya sehingga seluruh garis tulisan berwarna hijau tetap dapat diekstraksi tanpa terpotong.

Praktikum No. 2

IMPORT LIBRARY

```
In [1]: import cv2
import matplotlib.pyplot as plt
import numpy as np
```

Disini saya memakai kode import cv2 untuk memanggil library OpenCV. Library ini saya gunakan karena OpenCV adalah pustaka utama untuk pengolahan citra digital. Fungsinya nanti akan saya pakai untuk membaca file gambar, mengubah warnanya, melakukan thresholding, resize, dan lain-lain. Kemudian saya juga menulis import matplotlib.pyplot as plt. Matplotlib ini saya gunakan untuk menampilkan gambar di Jupyter Notebook, lalu saya mengimpor library NumPy dan memberi alias np. NumPy adalah pustaka untuk komputasi numerik.

Membaca Gambar

```
In [3]: img = cv2.imread('backlight.jpeg')
```

Disini saya memakai kode cv2.imread('backlight.jpeg') untuk membaca file gambar yang bernama backlight.jpeg. Fungsi imread ini berasal dari library OpenCV yang sudah saya import sebelumnya. Tanda kutip di dalam ('backlight.jpeg') menunjukkan bahwa nama file adalah string. File backlight.jpeg ini harus berada di folder yang sama dengan file notebook Python saya, kalau tidak, gambarnya tidak akan terbaca dan akan menghasilkan error. Hasil dari cv2.imread() ini saya simpan ke dalam variabel bernama img.

```
In [4]: [baris, kolom] = img.shape[:2]
```

Kode ini berfungsi untuk memperoleh ukuran citra, tepatnya tinggi dan lebar citra, dalam satuan piksel. Variabel img adalah array NumPy hasil pembacaan file gambar menggunakan cv2.imread. Atribut .shape pada array tersebut mengembalikan sebuah tuple yang berisi dimensi array. Untuk citra berwarna (RGB/BGR), img.shape memberikan tiga nilai: (tinggi, lebar, jumlah_kanal). Sementara untuk citra grayscale, hanya dua nilai: (tinggi, lebar). Dengan menuliskan [:2], kita mengambil dua elemen pertama dari tuple, yaitu tinggi dan lebar. Nilai tinggi disimpan ke variabel baris, dan lebar ke kolom.

KONVERSI TO GRAYSCALE

```
In [5]: rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Baris pertama mengubah citra dari format BGR (Blue Green Red) yang menjadi standar penyimpanan OpenCV ke format RGB (Red Green Blue) yang digunakan oleh pustaka visualisasi matplotlib. Tanpa konversi ini, citra akan tampil dengan kanal merah dan biru yang tertukar saat ditampilkan menggunakan imshow. Oleh karena itu, variabel rgb khusus disiapkan untuk keperluan penampilan citra asli agar warna tetap sesuai dengan aslinya.

Baris kedua mengonversi citra asli ke dalam ruang warna skala keabuan (grayscale). Proses ini dilakukan dengan menerapkan bobot yang meniru kepekaan mata manusia: 0,299 untuk kanal merah, 0,587 untuk hijau, dan 0,114 untuk biru.

Show Citra Asli Dan Gray

```
In [6]: fig, axs = plt.subplots(1, 2, figsize=(12, 5))
axs[0].imshow(rgb)
axs[0].set_title('Citra Asli (Backlight)')
axs[0].axis('off')

axs[1].imshow(gray, cmap='gray')
axs[1].set_title('Grayscale')
axs[1].axis('off')
plt.show()
```



Baris pertama, `plt.subplots(1, 2, figsize=(12, 5))`, berfungsi menciptakan sebuah kanvas dan satu baris yang memuat dua sumbu (axes) secara horizontal. Parameter `figsize=(12,5)` mengatur lebar kanvas 12 inci dan tinggi 5 inci agar gambar cukup besar dan mudah diamati. Objek `axs` adalah daftar yang berisi dua sumbu. (`axs[0]`) digunakan untuk menampilkan citra asli yang sudah dikonversi ke format RGB melalui `imshow(rgb)`. Metode `set_title` memberi judul 'Citra Asli (Backlight)' agar mudah dikenali. Kemudian `axis('off')` menyembunyikan semua penanda sumbu, seperti garis tepi dan angka koordinat sehingga hanya gambar yang terlihat.

(`axs[1]`) menjalankan prosedur serupa, namun untuk citra grayscale. Parameter `cmap='gray'` pada `imshow` memastikan bahwa nilai intensitas 0–255 diinterpretasikan sebagai gradasi hitam-putih, bukan. Judul diatur menjadi 'Grayscale'. Di akhir, `plt.show()` memerintahkan Matplotlib untuk merender dan menampilkan kanvas yang telah diisi.

OPERASI PIKSEL GRAY PENCERAHAN

```
beta = 50
citra_cerah = np.zeros((baris, kolom))

for x in range(baris):
    for y in range(kolom):
        gyx = gray[x, y] + beta
        if gyx > 255:
            gyx = 255
        citra_cerah[x, y] = gyx
citra_cerah = citra_cerah.astype(np.uint8)
```

Variabel `beta = 50` mendefinisikan besar pergeseran intensitas. Semakin besar nilai `beta`, semakin terang citra yang dihasilkan, karena seluruh piksel digeser ke arah putih (nilai 255). Matriks penampung `citra_cerah` dibuat dengan ukuran dua dimensi (`baris, kolom`) menggunakan `np.zeros`, yang menunjukkan bahwa citra hasil akan tetap berupa citra *grayscale* (satu kanal).

Struktur perulangan for x in range(baris): for y in range(kolom): memastikan setiap piksel diproses satu per satu. Di dalam perulangan, nilai intensitas asli gray[x, y] ditambahkan dengan beta, hasilnya disimpan sementara di variabel gyx. Untuk mencegah kelebihan nilai di atas 255, disisipkan pengecekan: jika gyx > 255, maka nilainya dikunci menjadi 255. Langkah ini penting karena piksel 8-bit hanya mampu menyimpan nilai 0–255; tanpa *clipping*, penjumlahan yang melebihi 255 akan menimbulkan *overflow* dan menghasilkan piksel gelap yang tidak diinginkan. Setelah diklip, nilai tersebut disalin ke matriks citra_cerah pada posisi yang sama. Baris terakhir citra_cerah = citra_cerah.astype(np.uint8) mengonversi tipe data matriks menjadi *unsigned integer* 8-bit, sehingga siap ditampilkan sebagai citra.

OPERASI PIKSEL GRAY KONTRAS

```

In alpha = 1.8
citra_kontras = np.zeros((baris, kolom))

for x in range(baris):
    for y in range(kolom):
        gyx = gray[x, y] * alpha
        if gyx > 255:
            gyx = 255
        citra_kontras[x, y] = gyx
citra_kontras = citra_kontras.astype(np.uint8)

```

Variabel alpha = 1.8 berfungsi sebagai faktor pengali kontras. citra_kontras dibuat dua dimensi dengan np.zeros((baris, kolom)) sebagai tempat menampung hasil operasi. Pemakaian array 2D menegaskan bahwa citra keluaran tetap satu kanal (grayscale). Selanjutnya, perulangan for x dan for y memproses setiap piksel secara individu. Nilai intensitas asli dari gray[x, y] dikalikan dengan alpha, disimpan di variabel gyx. Jika hasil perkalian melebihi 255, maka nilai tersebut dipotong (clipping) tepat ke 255 untuk mencegah overflow yang akan mengubah nilai besar menjadi kecil secara tidak terduga. Setelah aman, nilai tersebut ditempatkan ke citra_kontras[x, y]. Di akhir, konversi .astype(np.uint8) memastikan matriks bertipe data 8-bit unsigned integer

OPERASI PIKSEL BRIGHTNES & KONTRAS

```

In [106]: In alpha2 = 1.4
          beta2 = 35
          citra_gabungan = np.zeros((baris, kolom))

          for x in range(baris):
              for y in range(kolom):
                  gyx = alpha2 * gray[x, y] + beta2
                  if gyx > 255:
                      gyx = 255
                  citra_gabungan[x, y] = gyx
          citra_gabungan = citra_gabungan.astype(np.uint8)

```

Variabel alpha2 = 1.4 berperan sebagai faktor pengali kontras, sedangkan beta2 = 35 menjadi konstanta penambah kecerahan citra_gabungan dibuat dua dimensi (baris, kolom) sebagai penampung hasil pemrosesan. Struktur perulangan ganda memproses setiap piksel pada citra grayscale asli. Untuk setiap piksel, diambil nilai intensitas gray[x, y], kemudian dikalikan dengan

alpha2 dan ditambah beta2. Hasil perhitungan disimpan di variabel `gyx`. Guna menjaga agar tidak ada nilai yang melampaui batas atas representasi 8-bit, dilakukan clipping: jika `gyx > 255`, nilainya dipangkas menjadi 255. Langkah ini menjamin citra hasil bebas dari distorsi akibat overflow. Selanjutnya, nilai yang sudah aman ditempatkan ke dalam `citra_gabungan` pada koordinat yang sesuai. Akhirnya, konversi tipe data ke `uint8` menggunakan `.astype(np.uint8)`.

MENAMPILKAN SEMUA HASIL OPERASI

```
j:  fig, axs = plt.subplots(2, 2, figsize=(12, 10))

    # Grayscale Asli
    axs[0, 0].imshow(gray, cmap='gray')
    axs[0, 0].set_title('Grayscale Asli')
    axs[0, 0].axis('off')

    # Grayscale Dipercerah
    axs[0, 1].imshow(citra_cerah, cmap='gray')
    axs[0, 1].set_title(f'Dipercerah (+50)')
    axs[0, 1].axis('off')

    # Grayscale Dikontras
    axs[1, 0].imshow(citra_kontras, cmap='gray')
    axs[1, 0].set_title(f'Dikontras (x1.8)')
    axs[1, 0].axis('off')

    # Grayscale Gabungan
    axs[1, 1].imshow(citra_gabungan, cmap='gray')
    axs[1, 1].set_title(f'Gabungan')
    axs[1, 1].axis('off')

    plt.tight_layout()
    plt.show()
```

`fig, axs = plt.subplots(2, 2, figsize=(12, 10))` Membuat sebuah kanvas baru yang memuat grid subplot berukuran 2 baris $\times$ 2 kolom. Variabel `axs` sekarang adalah array 2×2 yang berisi empat objek axes, masing-masing akan ditempati satu citra. Parameter `figsize=(12, 10)` mengatur lebar 12 inci dan tinggi 10 inci agar seluruh subplot cukup besar.

`axs[0, 0]` (Grayscale Asli) Menampilkan citra gray menggunakan `imshow` dengan peta warna 'gray' agar array 2D dipetakan sebagai gradasi hitam-putih. Judul diatur menjadi 'Grayscale Asli' melalui `set_title`, dan sumbu dimatikan dengan `axis('off')` agar hanya gambar yang tampak.

`axs[0, 1]` (Hasil Pencerahan) Menampilkan `citra_cerah` yang telah diproses dengan $\beta = 50$. Judul 'Dipercerah (+50)' menunjukkan besar konstanta pencerahan yang diterapkan. Tampilan ini memperlihatkan efek penambahan intensitas, seluruh citra menjadi lebih terang bagian subjek yang semula gelap kini lebih tampak.

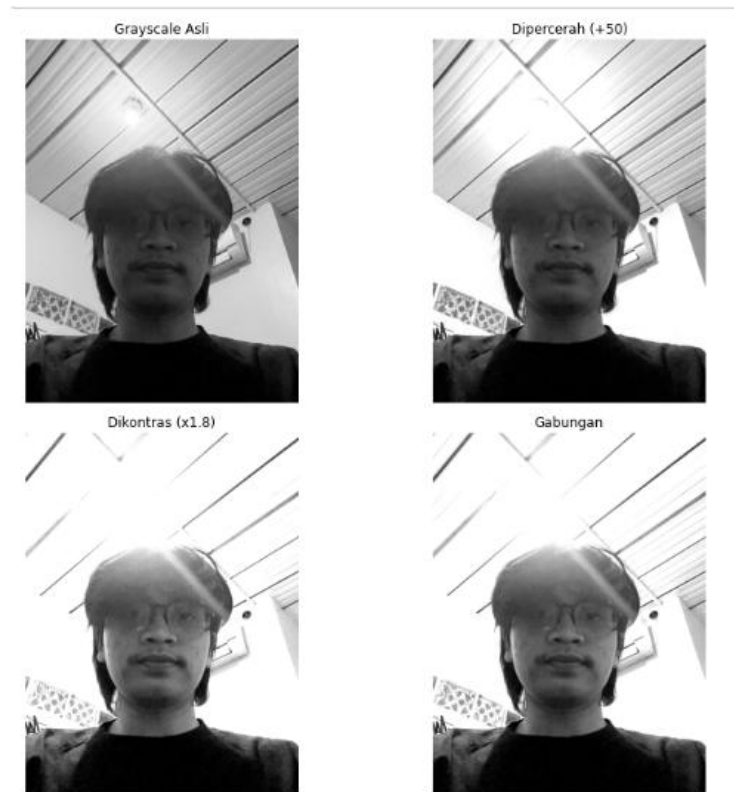
`axs[1, 0]` (Hasil Kontras) Menampilkan `citra_kontras` hasil perkalian dengan $\alpha = 1.8$. Judul 'Dikontras (x1.8)' mengindikasikan faktor pengali kontras. Di sini terlihat bahwa area terang bertambah terang, sedangkan area gelap relatif tetap, sehingga perbedaan intensitas semakin mencolok.

`axs[1, 1]` (Hasil Gabungan) Menampilkan `citra_gabungan`, yaitu hasil akhir dari kombinasi kontras dan pencerahan ($\alpha = 1.4$, $\beta = 35$). Judul 'Gabungan' menandakan bahwa ini adalah solusi lengkap untuk memperbaiki citra backlight, di mana detail subjek diangkat tanpa membuat latar terlalu terbakar.

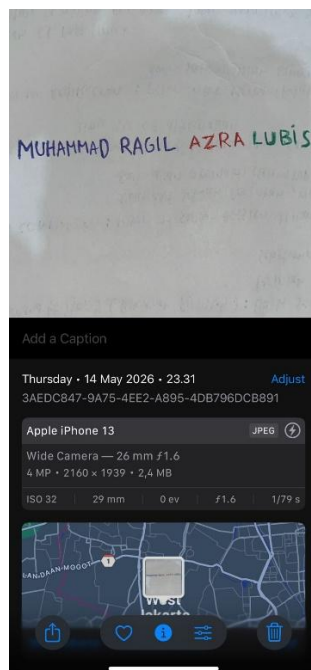
`plt.tight_layout()` Secara otomatis menyesuaikan jarak antar subplot agar tidak saling tumpang tindih, termasuk judul dan label. Hasilnya adalah tata letak yang rapi dan mudah dibaca.

`plt.show()` Merender seluruh figure dan menampilkannya di notebook atau jendela output.

OUTPUT



PELAMPIRAN BUKTI



BAB IV

PENUTUP

Berdasarkan landasan teori dan hasil praktikum yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Deteksi warna menggunakan ruang HSV** memberikan hasil yang lebih akurat dan tahan terhadap variasi pencahayaan dibandingkan thresholding langsung pada ruang RGB. Pemilihan rentang Hue yang tepat berdasarkan posisi warna pada roda warna (biru: 100–140, hijau: 35–90, merah: 0–10 dan 170–180) serta penentuan batas minimal Saturation dan Value berhasil memisahkan tinta berwarna dari latar putih yang memiliki saturasi rendah.
2. **Analisis histogram** memegang peranan penting dalam menentukan ambang batas. Histogram kanal merah, hijau, dan biru menunjukkan dua kelompok puncak: puncak tinggi dari latar putih dan puncak lebih rendah dari tinta berwarna. Pola ini memvalidasi bahwa pemisahan berbasis rentang dapat dilakukan dengan baik.
3. **Operasi pencerahan** dengan menambahkan konstanta beta berhasil mengangkat intensitas piksel di area gelap subjek, namun latar belakang ikut terang. **Operasi kontras** dengan faktor pengali alpha meningkatkan perbedaan antara area terang dan gelap, tetapi subjek yang gelap tetap kurang terlihat.
4. **Kombinasi pencerahan dan kontras** dalam persamaan linear $s = \alpha \cdot r + \beta$ memberikan hasil paling seimbang. Subjek yang semula gelap akibat efek backlight berhasil dinaikkan detailnya tanpa membuat latar belakang mengalami color burn yang berlebihan.
5. Secara keseluruhan, seluruh tahapan pengolahan citra mulai dari ekstraksi kanal warna, analisis histogram, deteksi warna berbasis HSV, hingga perbaikan citra backlight berhasil diimplementasikan menggunakan bahasa pemrograman Python dan pustaka OpenCV serta Matplotlib.

DAFTAR PUSTAKA

- Amrozi, Y., Haryanto, E., & Setiawan, A. (2024). *Deteksi warna dasar menggunakan metode thresholding HSV dengan OpenCV. Jurnal Neptunus: Asosiasi Riset Teknik Elektro dan Informatika Indonesia*, *12*(1), 1–10.
- Kartika, V. S., & Rochmatika, R. A. (2021). *Pelacakan obyek warna untuk mengontrol 2 axis servo dengan metode HSV filtering. JTET (Jurnal Teknik Elektro Terapan)*, *10*(1), 1–9.
- Lecca, M. (2023). *Backlight and spotlight image enhancement based on Von Kries model. SN Computer Science*, *4*(5), 680.
- Lubis, F. H., Syahrizal, M., Tampubolon, K., & Sinurat, S. (2021). *Peningkatan kontras menggunakan metode contrast limited adaptive histogram equalization pada citra paru-paru yang kecanduan rokok. JURIKOM (Jurnal Riset Komputer)*, *8*(1), 15–19.